

Capstone Part 2 Report: Metro Interstate Traffic Volume Analysis

1. Introduction

This project developed a reproducible Python workflow for analysing the Metro Interstate Traffic Volume dataset. The objective was to build on the exploratory analysis performed in Part 1 and create a structured data pipeline incorporating data validation, cleaning, feature engineering, visualisation, logging, automated testing and a simple command-line application.

The workflow was divided into separate Python modules so that the main processing tasks could be maintained and executed independently.

2. Data Preparation and Pipeline

The main data-processing workflow was implemented in `pipeline.py`. The pipeline loads the raw traffic dataset and validates the expected schema before further processing.

Data cleaning included removal of exact duplicate records and conversion of the `date_time` field into datetime format. Invalid temperature values were identified and treated as missing values, after which they were imputed using the median temperature. Logging was incorporated throughout the pipeline to record important processing events, warnings and the progress of the workflow.

After cleaning, the dataset contained 48,187 observations.

3. Feature Engineering

Feature engineering was implemented in a separate `feature_engineering.py` module.

Several variables were derived from the original data to facilitate traffic-pattern analysis. These included hour of day, day of week and a weekday/weekend indicator. Because hour is cyclical, sine and cosine transformations were also generated to represent the relationship between late-night and early-morning hours more appropriately.

Categorical weather conditions were one-hot encoded. Continuous temperature and cloud-cover variables were standardised. Traffic-volume categories of Low, Medium and High were also generated using data-driven tertile thresholds.

These engineered variables provide a structured dataset suitable for subsequent analysis and modelling.

4. Traffic Visualisation and Findings

Three visualisations were generated automatically and saved to the `figures/` directory: average traffic volume by hour of day, weekday versus weekend traffic volume by hour, and average traffic volume by day of week.

Traffic volume showed a strong relationship with time of day. Average traffic was lowest during the early morning hours, reaching its minimum at approximately 3 a.m. Traffic volume then increased sharply between approximately 5 a.m. and 7 a.m. A second increase occurred during the afternoon, with the highest average traffic volume occurring at approximately 4 p.m., at around 5,600 vehicles. Traffic subsequently declined throughout the evening.

Average traffic volume also increased progressively across the working week, from approximately 3,320 on Monday to a peak of approximately 3,680 on Friday, before falling sharply to approximately 2,820 on Saturday and 2,430 on Sunday.

The analysis also demonstrated a clear difference between weekdays and weekends. Average weekday traffic volume was approximately 3,557 vehicles compared with approximately 2,623 vehicles during weekends, making average weekday traffic about 36% higher. The weekday-versus-weekend hourly visualisation provides additional detail on how these differences vary across the day.

Overall, the findings indicate that temporal variables, particularly hour of day and weekday/weekend status, are important predictors of traffic demand.

5. Command-Line Application, Logging and Testing

A small command-line application (`traffic_app.py`) was developed to allow users to query the processed dataset interactively. The application supports queries for average traffic volume by hour of day, day of week and weather condition. Invalid user input is handled so that inappropriate values do not cause the application to terminate unexpectedly.

Python's `logging` module was used instead of `print()` for internal pipeline status and progress messages. Pipeline events are recorded in `pipeline.log`, providing a persistent record of processing activity. User-facing output in the command-line application is displayed using `print()`.

Automated tests were implemented in `test_pipeline.py` using `pytest`. Tests were created for schema validation, duplicate removal and datetime conversion. All three automated tests passed successfully during execution.

6. Conclusion

Part 2 transformed the exploratory traffic analysis into a more reproducible and modular Python project. The completed workflow integrates data validation and cleaning, feature engineering, visualisation, logging, automated testing and an interactive command-line application.

The analysis demonstrates clear temporal variation in traffic volume, with morning and afternoon increases and substantially higher average traffic on weekdays than weekends. The

modular structure provides a foundation for extending the project into more advanced modelling and deployment tasks in subsequent stages of the capstone.